

AULA 007 - EXPRESSÕES E CONTROLE

Slide 2:

Objetivo e ligação com o mundo real. O objetivo é entender como as linguagens avaliam expressões e controlam o fluxo, para que o código faça o que parece fazer. Três casos reais mostram o custo de não entender isso:

"goto fail" (Apple, 2014): uma linha duplicada pulava a verificação de assinatura.

Zune (2008): um laço infinito no último dia de um ano bissexto.

PHP com ==: hashes que começam com "0e" eram considerados iguais.

Os três são erros de estrutura de controle ou de avaliação de expressões, e não de lógica de negócio.

Slide 4:

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 • Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 • Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 • JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 • C

```
printf("%d\n", 3 > 2 > 1);
```

5 • Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 • JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula - Sebesta Cap. 7

Em Python, `-2 ** 2` dá -4 e `2 ** 3 ** 2` dá 512.

Em Java, `1 + 2 + "3" + 4 + 5` dá "3345".

Em JS, `"5" - 2` dá 3 e `"5" + 2` dá "52".

Em C, `3 > 2 > 1` dá 0.

Em Python, `x or "padrão"` dá "padrão".

No `==` do JS, o resultado é `true` `false` `true`.

	 main.py × +	 Console	 I/O	 AI Agent
	<pre>1 print(-2 ** 2, 2 ** 3 ** 2) 2</pre>	-4 512		

	 Main.java × +	 Console	 I/O	 AI Agent
	<pre>1 import java.util.*; 2 3 public class Main { 4 public static void main(String[] args) { 5 System.out.println(1 + 2 + "3" + 4 + 5); 6 } 7 } 8</pre>	3345		

	 index.js × +	 Console	 I/O	 AI Agent
	<pre>1 console.log("5" - 2, "5" + 2); 2</pre>	3 52		

OneCompiler Upgrade

C Main.c x +

1 `#include <stdio.h>`
2 `int main()`
3 `{`
4 `printf("%d\n", 3 > 2 > 1);`
5
6 `}`

Console I/O AI Agent

0

OneCompiler Upgrade

main.py x +

1 `x = 0`
2 `print(x or "padrão", 3 > 2 > 1)`
3

Console I/O AI Agent

padrão True

OneCompiler Upgrade

JS index.js x +

1 `console.log(0 == "", "" == "0", 0 == "0");`
2

Console I/O AI Agent

true false true

Slide 5:

Expressões aritméticas e questões de projeto. Quem projeta uma linguagem precisa decidir seis coisas: precedência, associatividade, ordem de avaliação dos operandos, efeitos colaterais, sobrecarga e mistura de tipos. Em $a + b * c$ com 3, 4 e 5, o resultado varia conforme a regra: 35 se for da esquerda para a direita e 23 com precedência. O APL avalia sempre da direita para a esquerda e o Lisp dispensa precedência com a notação prefixa. Entendi que a precedência é uma convenção de projeto e não uma lei.

Slide 6:

Precedência e associatividade na prática. A tabela compara Python, JS e Java:

`-2 ** 2` é `-4` em Python e erro de sintaxe em JS.

`1 + 2 + "3"` dá `TypeError` em Python e `"33"` em JS e Java.

Na divisão e no resto de negativos, Java e C truncam em direção a zero, enquanto Python arredonda para baixo.

A associatividade desempata operadores de mesma precedência. A maioria associa à esquerda, mas `**` associa à direita. Há exceções curiosas, como o `^` do Visual Basic e o `--2^2` do Excel, que dá 4. A regra prática é usar parênteses.

Slide 7:

Ordem dos operandos e efeitos colaterais. No exemplo do livro, `a = a + fun1()` em que `fun1` altera `a` para 17 pode dar 8 ou 20. Java garante a ordem da esquerda para a direita e dá 8. C e Go dão 20, porque a ordem não é definida. Fixar a ordem dá previsibilidade, mas atrapalha otimizações, e proibir efeitos colaterais é difícil em linguagens imperativas. Sem efeitos colaterais há transparência referencial, ou seja, `f(x) + f(x)` pode virar `2 * f(x)`.

Slide 8:

Operadores sobrecarregados. O mesmo símbolo pode ter vários significados, como o `+` de Java, que soma e concatena. O risco é prejudicar a legibilidade, como o `&` de C++, que é "E bit a bit" e "endereço de". C++, C#, Python e Rust permitem sobrecarga definida pelo usuário, e Java não. No exemplo em Python, `__add__` e `__mul__` fazem `v1 + v2` funcionar com vetores. Em Java, `'a' + 1` dá 98, mas `"" + 'a' + 1` dá "a1".

Slide 9:

Conversões de tipos. Distingui três ideias:

Alargamento: vai para um tipo maior, como `int` para `double`. Parece seguro, mas pode perder precisão, e 123456789 virou 123456792 ao ir para `float`.

Estreitamento: vai para um tipo menor e pode perder informação.

Coerção e cast: a coerção é a conversão implícita, feita pelo compilador. O cast é a conversão explícita, pedida pelo programador.

Go, Rust, ML e F# exigem conversão explícita. O perigo da coerção é esconder erros, como uma variável digitada errada que mesmo assim compila.

Slide 10:

Coerção implícita em JavaScript e PHP. Aqui está a coerção levada ao extremo. `"5" - 2` dá 3, `"5" + 2` dá "52", `[] + {}` dá "[object Object]" e `"b" + "a" + +"a" + "a"` dá "baNaNaN". O `==` não é transitivo, porque `0 == ""` e `0 == "0"` são true, mas `"" == "0"` é false. No PHP, `"0e123" == "0e456"` é true, o que explica o bug dos hashes MD5. A regra prática é usar `===` e `!==`.

Slide 11:

Expressões relacionais e booleanas. Em C não há um verdadeiro booleano, e 0 é falso e qualquer outro valor é verdadeiro. Por isso $3 > 2 > 1$ dá 0: primeiro $3 > 2$ vira 1, depois $1 > 1$ é falso. Em Python a comparação é encadeada e dá True, como na matemática. C, Python e JS aceitam qualquer valor no if, com 0, vazio e nulo contando como falso. Java, C#, Go e Rust exigem booleano.

Slide 12:

Curto-circuito. O resultado é determinado sem avaliar todos os operandos. É isso que protege buscas como `i < lista.length && lista[i] != chave` de um acesso fora do índice. Em Java, `&&` faz curto-circuito e `&` não, o que pode causar `NullPointerException`. Em JS, `?.` evita erro com null e `??` só substitui null ou undefined, enquanto `||` perde o valor 0. O cuidado é com efeitos colaterais, porque o `b++` em `(a > b) || (b++ / 3)` só acontece às vezes.

Slide 13:

Sentenças de atribuição. Em C, JS e Perl a atribuição é uma expressão e devolve um valor. Isso permite `while ((ch = getchar()) != EOF)` e `sum = count = 0`, mas também o erro clássico `if (x = y)`, que compila. Java e C# evitam o erro porque o if exige booleano. Python não trata atribuição como expressão, exceto pelo operador morsa `:=`. Go e Rust restringem ainda mais, e Python permite atribuição múltipla, como `a, b = b, a`. Nas funcionais, `val` e `let` dão nome a um valor que não muda.

Slide 14: Atribuição de modo misto. C converte em qualquer direção, então `int i = 3.7` vira 3 em silêncio e `char c = 300` vira 44. Java só permite alargamento e exige cast explícito para estreitar. Mesmo assim, `i += 3.7` compila por causa de um cast escondido. A armadilha clássica é `double taxa = acertos / total * 100`, que dá 0.0 porque a divisão inteira acontece antes do alargamento.

Slide 15: Estruturas de controle, quantas são necessárias? Sequência, seleção e iteração. Böhm e Jacopini (1966) provaram que elas bastam para qualquer algoritmo, então o goto é desnecessário. As outras estruturas (`for`, `switch`, `do`, `foreach`) existem por facilidade de escrita. O limite é a legibilidade, porque uma linguagem muito grande faz com que cada pessoa conheça só um subconjunto dela.

Slide 16:

O "else pendente". Em C, Java e C++, o else pertence ao if mais próximo que ainda não tem else, e a endentação é ignorada. Por isso o exemplo com `result` dá 1 e engana quem confia na endentação. Python e F# usam a endentação como estrutura. Perl, Go e Rust obrigam chaves. Em Rust e F#, `if` é uma expressão que devolve valor, e Python tem `x if c else y`.

Slide 17:

Seleção múltipla. O `switch` de C "cai" nos casos seguintes se não houver `break`, e no exemplo com `index = 1` executa `odd`, `even` e o erro do `default`. C# proíbe isso e Go só cai com `fallthrough`. O `switch` com `->` do Java 14+ não cai e pode ser expressão. O `match` de Rust é exaustivo, então faltar um caso é erro de compilação.

Slide 18: Laços controlados por contador. O for de C é flexível, com expressões opcionais e variável alterável no corpo, tanto que `i += 2` dentro do corpo faz imprimir 0 3 6 9. É praticamente um laço lógico disfarçado. O for de Python percorre um iterável, e alterar `i` no corpo não muda nada. Go só tem for, e as funcionais usam recursão.

Slide 19:

Laços lógicos, saídas e iteradores.

Pós-teste: o do-while executa ao menos uma vez, e isso é fácil de esquecer.

Zune: o laço quebrou porque, em dias `== 366`, nenhum ramo mudava dias. O laço precisa garantir progresso a cada volta.

break rotulado: sai de laços aninhados, como em Java.

JavaScript: `for...in` percorre as chaves e `for...of` percorre os valores. Somar com `for...in` dá "0012".

Slide 20:

Desvio incondicional, o goto. O goto é a sentença mais poderosa e por isso a mais perigosa.

Dijkstra (1968) publicou "Go To Statement Considered Harmful". No "goto fail", a segunda linha `goto fail`; fica fora do if (não há chaves), então desvia sempre. A verificação da assinatura nunca roda e err continua 0, ou seja, "sucesso". Chaves obrigatórias e o -Wall do gcc teriam pego o bug. `break`, `continue` e `return` são gotos restritos que melhoram a legibilidade.

Slide 21:

Comandos protegidos de Dijkstra. Cada ramo tem uma guarda. Todas as guardas são avaliadas e, se várias forem verdadeiras, a escolha é não determinística. Se nenhuma for, é erro no if ou fim do laço no do ... od. No exemplo do máximo, se `x == y`, qualquer ramo serve. É a base do CSP de Hoare (1978) e do select de Go, que sorteia entre canais prontos. No experimento com 1000 repetições, o resultado ficou perto de 505 a 495.

Slide 25:

Síntese. A tabela resume o que vimos: para cada tema (precedência, operandos, conversões, booleanos, atribuição, seleção, laços e desvio) há uma armadilha e uma forma de as linguagens ajudarem. Linguagens modernas como Go, Rust, Kotlin e Swift aprenderam com os erros de C: exigem booleano no if, chaves obrigatórias, conversão explícita e switch sem queda. Cada restrição elimina uma classe de bugs ao custo de um pouco de facilidade de escrita, o trade-off de confiabilidade contra escrita do Cap. 1.